Library Management System: Software Requirements Specification

Project name on GitHub: Taskly Repository: https://github.com/kanishka0606-c/taskly Live site: https://kanishka0606-c.github.io/taskly/ Version: 1.0

1. Introduction

1.1 Purpose

This document describes the requirements of the Library Management System, a web application for managing a small library. It is written for the course instructor and for anyone who wants to understand what the system does.

1.2 Scope

The system lets library staff register and log in, manage a catalogue of books, issue books to members, take them back, and view registered members. It is a simple web application built with the MERN stack (MongoDB, Express, React, Node.js). It does not handle fines, reservations, barcode scanning, or separate admin and member roles.

1.3 Definitions

- MERN: MongoDB, Express, React, Node.js.
- JWT: JSON Web Token, used to keep a user logged in.
- API: the server interface the web page uses to read and save data.
- SRS: Software Requirements Specification.

1.4 Overview

Section 2 describes the product in general. Section 3 lists the specific requirements. Section 4 describes the data and Section 5 the design constraints.

2. Overall Description

2.1 Product Perspective

The system is a standalone client-server web application. The React client runs in the browser and talks to an Express API. The API stores data in a MongoDB Atlas database.

2.2 Product Functions

- Register and log in.
- Add, edit, and delete books.
- Issue a book to a member and return it.
- View all books, available books, and issued books.
- View all registered members.

2.3 User Classes

There is one user class: a logged-in user (library staff). Any logged-in user can manage books and members' issues. There are no separate admin and member roles in this version.

2.4 Operating Environment

- Client: any modern web browser.
- Server: Node.js v24 with Express 5.
- Database: MongoDB Atlas, accessed with Mongoose 9.
- Client hosting: GitHub Pages.

2.5 Assumptions and Dependencies

- The user has an internet connection.
- The MongoDB Atlas database and the API server are running.
- Secrets (database string and JWT secret) are stored in a private .env file and are not uploaded to GitHub.

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement
FR1	The system shall let a new user register with name, email, and password.
FR2	The system shall reject registration if the email is already used.
FR3	The system shall let a registered user log in with email and password and receive a token.
FR4	The system shall require a valid token for every function except register and login.
FR5	The system shall let a logged-in user add a book.
FR6	The system shall let a logged-in user edit a book's details.
FR7	The system shall let a logged-in user delete a book.
FR8	The system shall list all books, available books, and currently issued books.
FR9	The system shall let a logged-in user issue an available book to a member.
FR10	The system shall let a logged-in user return an issued book, making it available again.
FR11	The system shall list all registered members.
FR12	The system shall let a user log out.

3.2 External Interface Requirements

User interface. A plain browser interface with screens for login, register, books, and members.

API interface.

Method	Route	Purpose
POST	/api/auth/register	Create an account
POST	/api/auth/login	Log in and get a token
GET	/api/auth/me	Get the logged-in user
GET	/api/books	List all books
POST	/api/books	Add a book
PUT	/api/books/:id	Edit a book
PUT	/api/books/:id/issue	Issue a book to a member
PUT	/api/books/:id/return	Return a book
DELETE	/api/books/:id	Delete a book
GET	/api/members	List all members

Software interface. The server uses Mongoose to communicate with MongoDB Atlas.

3.3 Non-Functional Requirements

- **Security:** Passwords are hashed with bcryptjs and never stored as plain text. Login uses JSON Web Tokens. The CORS setting allows only the known client address. Secrets are kept out of the repository.
- **Usability:** The interface is simple enough for a first-time user to add and issue a book without instructions.
- **Performance:** Normal actions should finish within a few seconds under light use.
- **Reliability:** Data is stored in MongoDB Atlas, so it is kept after a page refresh or server restart.
- **Portability:** The client works in current versions of common browsers.
- **Maintainability:** The server code is split into models, controllers, routes, and middleware.

4. Data Requirements

4.1 User

- name
- email (unique)
- password (hashed, not returned in responses)

4.2 Book

- title
- author
- availability status
- the member the book is issued to, when issued

(The exact field names are those used in the code in the server/src/models folder.)

5. Constraints

- Built as a course project with the MERN stack.
- Single role only; no admin and member separation.
- The free tiers of MongoDB Atlas and GitHub Pages are used.

6. Future Enhancements

- Separate admin and member roles.
- Due dates and fines for late returns.
- Search and filtering of books.
- Email reminders for overdue books.